IMPERIAL

Imperial College London
Department of Mathematics

Beyond Binary Churn: Early Prediction of Subscription Renewal Trajectories

Jesper Phillips

CID: 06012484

Supervised by Dr Marina Evangelou

August 2026

Submitted in partial fulfilment of the requirements for the
MSc in Machine Learning and Data Science at Imperial College London

The work contained in this thesis is my own work unless otherwise stated.

Signed: Jesper Phillips Date: August 17th 2026

Acknowledgements

Thank you to Dr Marina Evangelou for her guidance as my advisor throughout this dissertation project.

I would also like to thank my classmates, who have acted as sparring partners throughout the two years of this programme.

Lastly, thank you to my Mum and Dad, my sister Annika, my girlfriend Hedda, and all my friends who have listened to my rants and supported me throughout the dissertation work.

Abstract

Customer churn is often looked at as a binary outcome, customers who leave vs customers who remain. This dissertation instead models subscription renewal as a four-class behavioural trajectory consisting of CHURN, CONTRACTION, STABLE, and EXPANSION. Using the WSDM–KKBox dataset we construct a 6.1 million user-expiry event dataset, with 3.78 million events retained for modelling after applying a pre-expiry activity requirement. Renewal outcomes are defined based on transaction status and changes in actual listening volume at expiration.

We compare direct multiclass classification, two-stage classification, regression-based methods, and sequence-based deep-learning models. Using Macro-F1 and five repeated user-grouped holdouts we evaluate the headline performances. A tuned three-model deep-learning ensemble that combines static customer information and daily listening sequences was the strongest model, achieving a Macro-F1 of 0.5128 ± 0.0011. This surpassed Direct XGBoost by a mean paired difference of 0.0736 and the tuned two-stage classifier by 0.0742, with both differences remaining significant after Holm correction.

We also evaluate prediction lead time, by rebuilding model inputs and training at early pre-expiration cutoffs. At 14 days before expiry, the deep-learning cutoff model retained 91.8% of its at-expiry Macro-F1, while larger declines occurred at 30 and 60 days. Thus we show that post-renewal behavioural trajectories contain predictable structure beyond binary churn, and that renewal-trajectory predictions can be made ahead of the actual subscription expiration.

Contents

1. Introduction

1.1. Motivation

In commercial subscription-based businesses a central problem is understanding renewals. Largely this is often looked at as churn prediction, understanding if a user leaves after the subscription ends.

However, churn is only one part of the renewal story. A customer renewing but reducing usage is not similar to a renewing customer that increases engagement. From a binary perspective they are the same, but commercially they are very different outcomes. Lower engagement after renewal could lead to churn, reduce upsell potential, or indicate product value dropping.

This dissertation studies renewals as a multi-class trajectory, not a pure binary churn problem. Instead of answering if a user is churning, we attempt to predict churn, contraction, stable renewals, and expansions.

By taking this approach a company would be able to make more targeted commercial decisions. A predicted contraction may need a different response to a predicted expanding customer. The outcome classes in this dissertation measure changes in user listening volume rather than direct commercial value, but the same general approach could also be applied in settings where contraction and expansion are linked more directly to commercial value, such as changes in the number of licences purchased at contract renewal.

The dataset used is a KKBox prediction challenge (Kaggle, 2017), which provides user, usage, and transactional data from KKBox, a music listening service similar to Spotify. Originally the competition was binary churn, but we reconstruct user expirations and define a four-class renewal outcome using membership data and user behaviour. This makes it a behavioural supervised multi-class classification problem.

1.2. Research Gap and Related Work

Most customer churn work treats renewals as binary, either the customer renews or it does not (de Lima Lemos et al., 2022; Imani et al., 2025). Critically the binary classification does not retain information of what happens post renewal.

There is related work on broader behavioural patterns and changes in user engagement. However, these are often more general prediction tasks and do not examine behaviour around a specific point in time, such as a subscription renewal. The reviewed work generally does not combine a specific subscription-expiry event with a multi-class post-renewal behavioural outcome. This creates a gap, as the commercial question is not only whether the outcome can be predicted, but whether it can be predicted early enough to respond to negative outcomes.

This dissertation differs from the majority of the reviewed churn literature in three connected ways. First, it models renewal as a four-class behavioural outcome rather than a binary churn outcome. Second, each observation is centred on a specific user-expiry event, with behaviour measured around that event. Third, the models are evaluated at different cutoff points before expiry, allowing us to examine the trade-off between prediction performance and the amount of time available to act.

1.3. Research Questions and Contributions

The dissertation analyses how well pre-expiry user behaviour can predict post-expiry renewal trajectories. We construct a user-event pair dataset from raw KKBox metadata and listening logs, and use this to answer the following research questions:

1. How well can pre-expiry usage data, transaction history, and user metadata predict four renewal trajectories: `CHURN`, `CONTRACTION`, `STABLE`, and `EXPANSION`?
2. Do sequence-based deep-learning models improve performance over simpler tabular machine learning methods?
3. How does the time-before-renewal affect prediction performance, and can the outcome be predicted early enough before expiry to be commercially useful?

This dissertation has threefold contributions. Firstly we construct a dataset from raw data, in which each observation represents a user-specific subscription expiry event, and define a four-class behavioural renewal outcome consisting of churn, contraction, stable, and expansion. Secondly we compare different modelling approaches for the classification task, including a direct multi-class classifier, a two-stage approach, a regression-based approach, and a sequence-based deep-learning model. Third, we analyse how predictive performance changes based on pre-expiry prediction at certain cutoffs in time, answering the commercial question of how early we can predict user behaviour.

2. Literature Review

In this literature review we examine the main areas relevant to the dissertation. This includes customer churn prediction, machine learning methods for churn, the relationship between usage and renewal behaviour, multi-state customer modelling, and previous work on the KKBox dataset. Together they establish the foundations for modelling methods used in this dissertation, and how current work differs through an event-centred, multi-class renewal outcome and prediction-timing analysis.

2.1. Customer Churn Prediction

A lot of existing literature treats customer churn and renewals as a binary outcome, for example in banking, credit card usage, or telecommunications. A paper by de Lima Lemos et al. (2022) studies churn prediction within financial institutions using supervised learning approaches. For customer retention this is related to the churn part of this dissertation, but the target remains binary and irrespective of a specific renewal time point.

Binary churn framing appears in other service industries. Momin et al. (2020) study customer churn prediction using machine learning methods in a service context, while Altinisik and Yilmaz (2019) study the prediction of customers intending to cancel credit-card subscriptions using machine learning algorithms. Deep-learning approaches are also increasingly used, with Vu (2024) proposing a combined deep-learning network model for customer churn prediction in banking. Although the modelling approaches differ, the target remains binary churn.

This binary formulation is the main limitation for the current dissertation. A binary churn model can separate customers who leave from customers who remain, but it cannot distinguish between different types of renewal. In a music subscription setting, a user who renews but listens far less after expiry may be at future risk, while a user who renews and increases listening may represent stronger product engagement. Both would be labelled as non-churn in a traditional churn model.

2.2. Machine Learning Methods for Churn

The churn prediction literature uses a wide range of methods. Logistic regression is normally used as a baseline, while tree-based models like random forest and gradient

boosting are also popular. In the banking churn study by de Lima Lemos et al. (2022), random forests are compared with methods such as logistic regression and k-nearest neighbours. This backs up using these methods for this dissertation.

The comparison to sequence-based deep-learning is motivated by recent papers exploring LSTM and CNN-based approaches to churn prediction. Imani et al. (2025) review current methods and identify both these, while also looking at XGBoost and LightGBM as commonly used options. Temporal customer behaviour has also been used in churn models. Gregory (2018) uses extreme gradient boosting with temporal data for customer churn prediction. Especially for customer engagement over time this becomes relevant, as recent behaviour might carry more weight than older behaviour. This provides support for using temporally structured customer behaviour as model input.

Class imbalance becomes extra important in the multi-class prediction model. Even in regular binary classifiers imbalance exists, with churn often being a small percentage of retention. Chawla et al. (2002) introduce SMOTE as a method for handling imbalanced classification by synthetically over-sampling minority classes. SMOTE itself is not used in this dissertation, as we instead use class weighting and evaluate using Macro-F1, balanced accuracy, and per-class performance metrics.

2.3. Usage, renewal, and contractual behaviour

Although normally a binary model, some related work looks into the relationship between usage and churn. Ascarza and Hardie (2013) look into a joint model between usage and churn for contracts. This recognises that customer activity and churn are connected. A customer who reduces usage may be more likely to churn later, while high usage may indicate a stronger chance for retention.

As a dissertation we differ from this paper on the modelling target. While we use the relationship between usage and churn, we directly predict a class label with regards to a specific event in time. This work is relevant as it supports usage as an informative signal of customer outcome. This dissertation extends that by using changes in usage to define behavioural outcomes among customers that renew.

Renewal-focused work also exists outside consumer churn prediction. John et al. (2021) study contract renewal using machine learning to support improving renewal rates. This is closer to the commercial renewal setting than general churn prediction because the business problem is explicitly about renewal. Although here the outcome remains

whether the contract renews, and does not separate renewed contracts according to actual changes within the renewal.

2.4. Multi-state churn and richer customer states

From the perspective of multi-class modelling a paper by Dong et al. (2022) gets conceptually closer. They propose a multi-state customer churn analysis in an insurance setting, where customer behaviour is modelled over several possible states rather than as a simple churn/non-churn split. This shows that churn can be viewed as movement between customer states, and not just a binary exit event.

This paper takes a similar view to multi-states and recognises that customer behaviour has more depth than a binary label can offer. However, the states are defined differently. Dong et al. look at insurance coverage combinations, where class movement is between how many or what types of policies a customer has, and the analysis is not centred around specific expiry events. This dissertation instead focuses on behavioural outcomes after a specific event, where states are defined by post-expiry listening behaviour around a subscription event.

2.5. The KKBox churn challenge

Solutions to the KKBox churn prediction challenge are naturally relevant, as they use the same raw data as this dissertation. The original WSDM–Kaggle challenge was a binary churn prediction after expiration (Kaggle, 2017). Wang et al. (2018) present a practical pipeline with stacking models for the KKBox churn prediction challenge, using feature engineering and ensemble methods to predict the original churn target. Gregory (2018) also uses temporal features and XGBoost in the WSDM churn challenge context.

These papers provide a foundational idea that the KKBox data is suitable for class modelling, and that strong tabular models can use the transaction and listening logs for accurate predictions. However they use the original binary churn outcome, while we reconstruct the expiry events into a new four-class target using both transactional renewal and post-expiry listening behaviour. The same raw data are therefore used to answer a different type of prediction, focusing on the direction of customer behaviour after a specific time event.

2.6. Research Gap

From the reviewed literature we identify three main gaps in research. First, most churn prediction studies use a binary outcome and therefore do not distinguish between different post-renewal behaviours. Second, research linking customer usage and churn shows that behavioural information is relevant, but the reviewed work does not generally centre the prediction task on a specific subscription-expiry event with behaviour measured around that point. Third, multi-state customer models demonstrate that richer customer outcomes can be modelled, but the reviewed work does not combine these states with post-expiry consumption changes and an explicit analysis of prediction lead time.

This dissertation addresses these gaps as we look at subscription expirations, transactional information, and actual usage behaviour. With these we define an observation as a user-expiration event, and the prediction target contains four outcomes: `CHURN`, `CONTRACTION`, `STABLE`, and `EXPANSION`. Thus we have a more difficult prediction problem than binary churn, but retain information about the direction of behaviour among renewed customers.

The dissertation also adds an explicit timing dimension. The models are evaluated at several cutoffs before expiry, with the most recent information withheld and all time-varying predictors rebuilt at the relevant prediction point. This allows us to examine the trade-off between predictive performance and the amount of lead time available before the renewal event.

3. Data and Methods

3.1. Data and Prediction Problem

3.1.1. Unit of Observation

This section is covering how the raw KKBox data is made into an event-user pair level dataset used for the actual modelling task. Thus we change the unit of observation from a single user to a user-expiry event (m, E), where m is the anonymised user identifier (`msno` in the actual data), and E is the membership expiration date.

A single user can appear multiple times, as users resubscribe during the time-frame of the dataset. This is both an important feature and an evaluation detail. At the same

time training-test splits can get messy if we allow training on users that then reappear in test sets. For each specific event, post-expiration data is used to label the outcome, but model input is pre-expiration only.

3.1.2. Raw Data Sources

The raw data used is the KKBox Churn prediction challenge from WSDM–Kaggle (Kaggle, 2017). KKBox is a consumer-level music streaming service, and the original task was to predict whether a user would churn or not at a given membership expiration date. We use the same raw data but do not use the original target labels and rather construct our own. The data is split into transactional data, listening logs and member data.

The transactional data comes in two files, `transactions.csv` and `transactions_v2.csv`. A transaction record contains the anonymised user identifier `msno`, the transaction date, the resulting membership expiry date, payment method, plan length, listed price, actual amount paid, auto-renewal status, and cancellation indicator (pre-expiration choice of cancelling). These fields are the time anchor of our dataset, with the membership expiration date working as the cutoff for pre/post behaviour.

Listening logs at a user level are given through `user_logs.csv` and `user_logs_v2.csv`. They are daily user-level logs, containing the date, total seconds listened, the number of unique songs, and counts of songs played to different completion thresholds. In the tabular models, we aggregate these logs into behavioural summaries. In the sequence models, we use the same logs to construct a daily pre-expiry sequence for each expiry event. The member data, `members_v3.csv`, contains city, age, gender, registration method, and registration date. Table 1 summarises the raw data sources and their role in the dissertation.

Table 1.: Overview of the raw KKBox data sources used in the dissertation.

Raw data source	**Unit**	**Role in the dissertation**
`transactions.csv`, `transactions_v2.csv`	User transaction	Used to construct expiry events, renewal labels, and transaction predictors.
`user_logs.csv`, `user_logs_v2.csv`	User-day	Used to construct listening-based labels, behavioural features, and sequence inputs.
`members_v3.csv`	User	Used to add demographic and registration predictors.

3.1.3. Event Construction and Cohort Window

The transaction files are concatenated and duplicate transaction records are removed. We then select transactions whose `membership_expire_date` falls within the cohort window. For each unique pair of `msno` and membership expiry date, we keep the latest associated transaction whose `transaction_date` is on or before that expiry date. This ensures that the event-defining transaction and its cancellation or auto-renewal fields were known at E. This gives one row per expiry event, with the membership expiry date renamed to E. The pre-event transaction fields are retained as candidate predictors, while later transaction information is used only for renewal outcome construction.

A six-month window is chosen to label and do the modelling off, with earlier data being retained for historical features. Thus we chose a timeframe of 2016-08-01 to 2017-01-31 to label renewal events. This six-month window gives 6,133,147 expiry events across 1,507,727 unique users. The user log data cover the period needed to construct both the pre-expiry and post-expiry windows around these events.

For any given E, we define a 30-day pre and post expiration window. Thus pre expiration is defined as $E - 30$ to $E - 1$, while the post-expiry window covers days $E + 1$ to $E + 30$. These two windows are what we use to construct the renewal outcome labels. Importantly the model features themselves are not limited to this 30-day pre-expiry window, and usage/transaction data stretching back to 2015-01-01 (for users that have that much history) is used for the actual modelling.

3.1.4. Renewal Outcome Construction

The outcomes for this dissertation are self-derived based on transaction and listening data. The final modelling classes are: `CHURN`, `CONTRACTION`, `STABLE`, and `EXPANSION`. In addition to this there is an `INSUFFICIENT_DATA` class which is excluded from model training.

Any user that does not have at least 7 active listening days during the pre-expiry window is labelled as `INSUFFICIENT_DATA`. These events are excluded entirely from the prediction work. This is done because the movement label compares post-expiry listening volume to a pre-expiry baseline. If the pre-expiry baseline is based on too few observed listening days, the ratio becomes noisy and the event is not considered reliable enough for supervised modelling.

For the remaining events the first step is to determine renewal. This is done if the customer either has a paid transaction within 30 days of the expiration date, or has renewed early before E and the transaction has already extended their membership beyond E. Specifically, a forward renewal is defined as a transaction in the interval $[E, E+30]$ where `actual_amount_paid > 0` and `is_cancel = 0`. An early renewal is defined as a non-cancelled transaction dated on or before E where the resulting `membership_expire_date > E`. If neither condition is satisfied, the event is labelled as `CHURN`. Thus churn is a true transactional label, not determined by actual usage.

For renewed users, we define a listening volume ratio:

$$R_i = \frac{\texttt{post\_total\_secs}_i}{\texttt{pre\_total\_secs}_i}$$

where `pre_total_secs` is total listening seconds in the 30 days before expiry and `post_total_secs` is total listening seconds in the 30 days after expiry. A modelling decision was made here on whether the ratio should incorporate active listening days, or pure volume only. In the end we conclude that usage is listening volume, and this becomes the clearest signal for contraction/expansion.

Any renewed user is then split into contraction, stable, and expansion using the listening ratio. Here we apply thresholds of 0.5 and 1.5. A renewed user with $R_i < 0.50$ is labelled `CONTRACTION`, while a renewed user with $R_i > 1.50$ is labelled `EXPANSION`, meaning a substantial shift in listening volume is needed to not be a stable renewal. Any event with $0.50 \leq R_i \leq 1.50$ is labelled `STABLE`. We chose the threshold of 50% movement through a threshold sweep which calibrated `STABLE` to approximately 60% of the core cohort.

Two edge cases required specific labelling decisions. First, some users renewed but recorded no post-expiry listening activity. We labelled these events as `CONTRACTION`, since the subscription remained transactionally active despite usage falling to zero. Second, some users labelled as `CHURN` still recorded limited listening after expiry. We examined these cases separately. Of the 160,535 `CHURN` events, 76,966 (47.9%) had non-zero post-expiry activity, but this activity was generally minimal: the median was one active day and 816 seconds, and these events accounted for only 0.3% of total post-expiry listening volume. Their median hypothetical post/pre listening ratio was 0.01, and within the predefined 30-day grace window the transaction audit found no missed qualifying paid, non-cancelled renewals. We therefore retained the transactional `CHURN` label and

interpreted the observed pattern as consistent with limited residual listening rather than actual renewal misclassification.

We do not apply any post-expiration activity filter. If a user has enough activity to get through the pre-expiration filter, but then falls below that activity level post-expiration, this is a genuine contraction and would be a design flaw to filter out.

The full labelling rules are:

1. If `pre_n_days < 7`, assign `INSUFFICIENT_DATA`.
2. Else if the user has neither a paid, non-cancelled renewal in $[E, E+30]$ nor a non-cancelled transaction dated on or before E which extends membership beyond E, assign `CHURN`.
3. Else if `post_total_secs = 0`, assign `CONTRACTION`.
4. Else compute $R_i = \texttt{post\_total\_secs}_i / \texttt{pre\_total\_secs}_i$.
5. If $R_i < 0.50$, assign `CONTRACTION`.
6. If $R_i > 1.50$, assign `EXPANSION`.
7. Otherwise, assign `STABLE`.

A flowchart of the labelling logic is displayed in Figure 4 in Supplementary Section A.1.

After applying this logic, 3,784,500 events remain in the core modellable dataset labelled into the classes `CHURN`, `CONTRACTION`, `STABLE`, and `EXPANSION`. The remaining 2,348,647 events are labelled `INSUFFICIENT_DATA` and are excluded from the supervised classification experiments.

3.1.5. Feature Engineering

We engineer features from the dataset using only available data before the given expiration event. Post-expiry information is used for the outcome labelling, but cut off before any data is passed to the model inputs. The final dataset includes behavioural listening features, transaction features, historical subscription features, demographic covariates, horizon features, and daily sequences.

The 30-day pre-expiry listening window gives the basic behavioural predictors: `pre_total_secs`, `pre_n_days`, and `pre_rate`. These represent the most recent engagement level. Transaction level features then describe the state of the subscription right before expiration, including listed price, actual amount paid, payment plan length, payment method, auto-renewal status, and cancellation indicator.

Historical transaction features are computed from transactions strictly before the expiry date E. These include the number of historical transactions, mean historical price, median historical amount paid, cancellation rate, auto-renewal rate, modal payment method, tenure in days, and recency of the latest historical transaction. We also create prior-renewal history features, including `n_prior_renewals` and `prior_renewal_rate`.

We also include behavioural features that compare the user's historical volume to the most recent. We compute `obs_secs_per_active_day`, the user's average listening seconds per active day over all observed listening days before E. The feature `recent_vs_life_ratio` is then defined as:

$$\texttt{recent\_vs\_life\_ratio}_i = \frac{\texttt{pre\_rate}_i}{\texttt{obs\_secs\_per\_active\_day}_i}$$

Thus we get a signal if the user's most recent listening volume is above or below their long-term baseline. The baseline uses whatever data is available, capped at 2015-01-01, so although it does have a significant history tail, it may not capture the entire user lifetime.

Member-level demographic features are merged from `members_v3.csv`. These include city, age, gender, registration method, and registration date. Some members have missing data, like age or city. These are not filtered out and remain in the final data.

For the time-before-renewal experiments, we create horizon features. Given a horizon, $h \in \{1, 7, 14, 30, 60\}$, we compute a 30-day observation window ending h days before E. These features are stored as `h1_*`, `h7_*`, `h14_*`, `h30_*`, and `h60_*`. The following time-before-renewal experiment uses decision cutoffs $c \in \{0, 7, 14, 30, 60\}$, where $c = 0$ represents prediction at expiry and $c > 0$ represents prediction using only information available by $E - c$.

For the deep-learning models we construct a daily sequence panel. Using 60 days of history we construct per active user day a panel storing `total_secs`, `num_100`, `num_unq`,

and num_25. The sequence modelling notebook pivots this panel into a tensor of shape $(n, 60, 4)$ and zero-fills missing days.

While doing initial exploratory analysis we examined relationships between engineered numeric features. This was done using Spearman rank correlations on a 200,000 event sample of the core cohort. We also looked at class-conditional distribution of certain features. The analysis was used to understand the engineered feature set, more-so than as a mechanical feature-selection rule.

Table 2 summarises the main feature groups used in the modelling pipeline.

Feature group	Example variables	Type	Source and timing
Event and indexing fields	msno, E, expiry_month	Identifier, date	Transactions; event construction
Pre-expiry listening	pre_total_secs, pre_n_days, pre_rate	Continuous, count	User logs; before prediction cutoff
Transaction state	pre_amount_paid, payment_plan_days, is_auto_renew, is_cancel	Continuous, count, binary	Transactions; before prediction cutoff
Transaction history	n_hist_txns, tenure_days, recency_days, cancel_rate	Continuous, count	Transactions before prediction cutoff
Prior renewal history	n_prior_renewals, prior_renewal_rate	Continuous, count	Transactions before prediction cutoff
Listening baseline	obs_secs_per_active_day, recent_vs_life_ratio	Continuous	User logs before prediction cutoff
Member covariates	city, bd, gender, registered_via	Categorical, continuous	Member table; static
Horizon features	h1_*, h7_*, h14_*, h30_*, h60_*	Continuous, count	User logs; horizon-specific
Sequence features	Daily total_secs, num_100, num_unq, num_25	Sequence	User logs; 60 days before cutoff

Table 2.: Main feature groups in the engineered expiry-event dataset.

3.2. Modelling Methods

In this section we cover the different modelling approaches used to predict the renewal outcome defined in Section 3.1. The models' purpose is to predict one of the 4 renewal

outcome classes:

$$\hat{y}_i \in \{\texttt{CHURN}, \texttt{CONTRACTION}, \texttt{STABLE}, \texttt{EXPANSION}\}.$$

Our main comparison is between direct tabular classification, two-stage classification, regression-based modelling, and a deep-learning approach. All models are applied to the same event-level cohort and the same seed-specific user splits. Differences in performance thus come from modelling methods and the features in the approaches, and not the differences in evaluated users or outcome labels.

3.2.1. Baseline Models

To establish a modelling lower-bound we run some simple baseline models. The main one is a multinomial logistic regression classifier. This is simple, interpretable, and provides a decent comparison point for the non-linear models. The model estimates one set of class probabilities per event and assigns the class with the highest probability.

The logistic regression baseline uses the engineered tabular feature set. Numeric variables are imputed and standardised, while categorical variables are encoded before fitting the model. Class imbalance is handled using class weighting, so that minority classes such as `CHURN` and `CONTRACTION` are not ignored by the classifier.

We also do some naive dummy baselines, including a majority-class always `STABLE` classifier. There is also a class-prior baseline. These are not attempts at classification, but give interpretation context to the Macro-F1 scores used later.

3.2.2. Direct Multiclass Models

Our first main approach is a direct multi-class classifier. This method is the most direct we will apply, as the model is trained on the same target as final evaluation. As it is only one stage we also do not need to worry about error propagation between stages. On the other hand the model must learn two distinct decisions concurrently, firstly whether a user churns or not, and if not then how the listening volume changes.

The direct multi-class models use the engineered tabular feature set summarised in Table 2.

As mentioned in the literature review, gradient-boosted tree models have performed strongly in previous churn-prediction work (Gregory, 2018; Imani et al., 2025). We therefore compare several model families in an initial model sweep:

- **Logistic Regression**: a linear probabilistic classifier, included as a simple and interpretable baseline, consistent with its use in previous churn-prediction comparisons (de Lima Lemos et al., 2022).
- **Random Forest**: an ensemble of decision trees trained using bootstrap samples and random feature subsets, reducing the variance of an individual tree (Breiman, 2001).
- **HistGradientBoosting**: a histogram-based implementation of gradient boosting, based on the gradient-boosting framework (Friedman, 2001).
- **CatBoost**: a gradient-boosting method designed to handle categorical predictors directly (Prokhorenkova et al., 2018).
- **LightGBM**: an efficient gradient-boosted tree implementation using histogram-based learning and leaf-wise tree growth (Ke et al., 2017).
- **XGBoost**: a regularised gradient-boosted tree framework designed for scalable learning (Chen and Guestrin, 2016).

3.2.3. Two-Stage Classification

The second approach is to do a two-stage classifier. Where instead of directly classifying outcome, we first classify churn and renewal:

$$\hat{z}_i \in \{\texttt{CHURN}, \texttt{RENEWED}\}.$$

Then from predicted renewing customers the second stage will classify the type of renewal:

$$\hat{y}_i^{(2)} \in \{\texttt{CONTRACTION}, \texttt{STABLE}, \texttt{EXPANSION}\}.$$

Thus the final 4 classes will be the same as in method one, but with a different approach:

$$\hat{y}_i = \begin{cases} \texttt{CHURN}, & \text{if } \hat{z}_i = \texttt{CHURN}, \\ \hat{y}_i^{(2)}, & \text{if } \hat{z}_i = \texttt{RENEWED}. \end{cases}$$

This method is tested as churn is fundamentally different than a type of renewal, and the hypothesis is that the classifiers can focus on two different things and therefore increase accuracy. This is closer to how the business problem may be interpreted: first, will the customer renew, and second, if they renew, will their engagement decline, remain stable, or grow?

There is also a modelling advantage as each classifying question is simpler. First a binary problem, and then a 3 class problem. On the other side there is the risk of error propagation. The first classifiers errors are guaranteed to carry through to the end evaluation.

Both stages use the same engineered tabular feature set as the direct multiclass setup. LightGBM is used as the main model family for both stages, with logistic regression included as a baseline. This allows a fair comparison between a single four-class classifier and a decomposed hierarchical classification strategy. We evaluate both the standard argmax version of the two-stage model and a threshold-adjusted version, where the churn-renewal decision threshold is tuned using development-set out-of-fold predictions to improve Macro-F1.

3.2.4. Regression-Based Modelling

We also test an approach where stage 1 is done as a classification, but stage 2 is done with a regression. Two versions are used. The first directly predicts the post/pre listening ratio, while the second predicts post-expiry listening volume and then derives the listening ratio R_i based on the pre-expiry volume we already know. The predicted ratio is then used to label the class with the known 0.50 and 1.50 thresholds. This produces another approach investigating if regression produces better accuracy than direct classification.

3.2.5. Deep-Learning Models

The last modelling approach uses deep learning, in an attempt to capture non-linear patterns missed by the classical tabular methods. The sequence-based deep-learning methods preserve the daily structure of listening behaviour.

For each expiry event, the sequence input is defined over a fixed 60-day pre-event window:

$$X_i^{seq} = \{x_{i,t}\}_{t=1}^{T},$$

where $T = 60$ and $x_{i,t}$ contains the user's listening-log features on day t before the event date. The daily sequence features are total listening seconds, the number of songs played to completion, the number of unique songs, and the number of short plays. Missing listening days are represented as zero-valued rows. Static customer, transaction, aggregate listening, and horizon-based features are represented separately as X_i^{static}.

We test three deep-learning approaches. Firstly, a static multilayer perceptron (MLP) is trained on the previously used tabular dataset. This is the deep-learning baseline, as it does not use the sequential data. We also train a hybrid recurrent model using a gated recurrent unit (GRU) (Cho et al., 2014), which encodes the daily sequence. This sequence representation is concatenated with the static feature vector before classification. Lastly, an enhanced ensemble of three independently trained hybrid models is evaluated by averaging the class probabilities across the three models. The ensemble is evaluated using both a standard argmax decision rule and tuned class-specific decision weights.

The basic hybrid model can be generally written as:

$$\begin{aligned} h_i^{\text{seq}} &= \text{GRU}(X_i^{\text{seq}}), \\ h_i &= [h_i^{\text{seq}}, f(X_i^{\text{static}})], \\ \hat{p}_i &= \text{softmax}(g(h_i)). \end{aligned}$$

where h_i^{seq} is the learned sequence representation, $f(X_i^{static})$ is the static feature representation, and $g(\cdot)$ is the final classification head. The prediction remains a direct four-class classification problem.

The enhanced hybrid model uses a bidirectional GRU with attention pooling and combines the resulting sequence representation with static and horizon-based engagement features. Three independently trained versions of this model are combined by averaging their predicted class probabilities:

$$\hat{p}_{i,k}^{\text{ens}} = \frac{1}{M} \sum_{m=1}^{M} \hat{p}_{i,k}^{(m)},$$

where $M = 3$ represents the number of independently trained hybrid models.

For the tuned ensemble, the final prediction is instead:

$$\hat{y}_i = \arg\max_k \left(w_k \hat{p}_{i,k}^{\text{ens}} \right),$$

where w_k is a class-specific decision weight selected using development-set predictions to improve Macro-F1. These weights adjust the final class decision rule but do not change the trained neural-network parameters.

3.3. Experimental Design and Implementation

3.3.1. Data Splitting and Leakage Control

As the complete data contains multiple expirations per user, randomly splitting the train/test data could cause leakage by the same users being trained and tested on at different expirations. A model could learn persistent user-level patterns from one expiry event and then be evaluated on another event from the same user. To avoid this, all headline train-validation-test splits are grouped by `msno`. A user is assigned entirely to either the development data or the held-out test data.

For headline evaluations we use five seeded user-grouped holdout runs. In each run $\sim 15\%$ of users are assigned to the holdout set, and all events belonging to an assigned user are entirely assigned to holdout data. Remaining users are the development set for that run. For the tabular models, the selected model is then refitted using all development users and evaluated once on the untouched holdout. The same seed-specific user splits are used across modelling approaches.

Several leakage controls are applied during feature construction. All post-expiry variables are excluded from the model input matrix. This includes the renewal indicator, post-expiry transaction fields, post-expiry listening totals, post-expiry listening rates, the volume ratio, and other columns derived from the target construction process. These variables are kept in the engineered dataset for auditing and label construction, but are not passed to the models.

In addition to this, all historical features are created at a point in time. Transaction history features use only transactions with `transaction_date < E`. Prior-renewal features use only transactions before the current expiry event. The observed listening baseline

uses listening days before E. Horizon features use observation windows that end before the relevant prediction cutoff. Sequence features use only offsets -60 to -1, so no post-expiry sequence data is available to the deep-learning models.

Model preprocessing is fit using training or development data only. Numeric imputation and standardisation are fit without using held-out test rows. Categorical encodings are handled within the training pipeline, with missing or unseen categories treated explicitly. For the deep-learning models, the daily sequence channels are log-transformed and standardised using development data only.

3.3.2. Model Selection and Hyperparameter Tuning

All model selection is carried out on the development users only. The held-out users are not used for hyperparameter tuning, threshold tuning, model selection, or preprocessing decisions. Thus we ensure the final evaluation tests are entirely separate from any training and selection data.

For the tabular models we first do a model sweep over the six model families described in Section 3.2.2. This comparison uses three-fold user-grouped cross-validation on a label-stratified subsample of approximately 200,000 development events. The best-performing model families, LightGBM and XGBoost, are then tuned using randomised hyperparameter sweeps.

For each of the five repeated-holdout runs, LightGBM and XGBoost are re-tuned independently using 16 randomly sampled hyperparameter configurations and three-fold user-grouped cross-validation on a 200,000-event subsample drawn only from that run's development users. The configuration with the highest mean validation Macro-F1 is then refitted on all development users and evaluated once on that run's untouched hold-out. Consequently, there is no single final hyperparameter configuration shared across all five runs. The full search spaces are reported in Supplementary Section A.3.

The same LightGBM search space and development-only tuning procedure are used for the two-stage classifier and the regression-based models. The renewal stage is selected using renewal-versus-churn Macro-F1, while the movement classifier is selected using three-class Macro-F1 among renewed events. For the regression models, predicted continuous values are converted back into movement classes, and the hyperparameter search is ranked using the resulting movement Macro-F1 rather than regression error alone.

For models that support direct class weighting, we use balanced class weights as 63% of events are in the stable class. For the other models balanced sample weights are passed during training.

In the deep-learning section, training uses a validation partition drawn from the development data for early stopping and model selection. The loss function is class-balanced cross-entropy, and the optimiser is Adam or AdamW depending on the model version (Loshchilov and Hutter, 2017). The sequence channels are log-transformed and standardised using development data only. Numeric static features are imputed and standardised, while categorical features are represented using embedding layers. The best model state is selected using validation Macro-F1.

For the final three-model ensemble, a separate validation step is used to tune the class-specific decision weights. The weights are selected to optimise Macro-F1 without changing the trained neural-network parameters.

3.3.3. Time-Before-Renewal Experiments

For the time-before-renewal experiment, the cutoffs used are:

$$c \in \{0, 7, 14, 30, 60\}.$$

At $c = 0$, the model has access to all information available by the expiry event E. At each earlier cutoff, information from the final c days before E is withheld throughout both training and evaluation. The transaction features, transaction-history features, prior-renewal features, and listening features are rebuilt using only information available by $E - c$. For the deep-learning model, the daily listening sequence is shifted to end at the relevant cutoff. Each model is retrained separately at each cutoff.

For the deep-learning model, the numeric and categorical static features are also rebuilt using only information available at the relevant cutoff. The class-specific decision weights selected for the main at-expiry ensemble are held fixed across all cutoffs rather than re-tuned separately. This isolates the effect of removing recent information while keeping the final decision rule constant. Re-tuning the weights at every cutoff could partly compensate for the loss of recent data and would make it less clear whether changes in performance were caused by the earlier prediction point or by a newly optimised decision rule.

The headline at-expiry comparisons use five repeated user-grouped holdouts. The time-before-renewal analyses use the same underlying events and outcome labels across the tabular and deep-learning models, but apply different evaluation procedures. The tabular experiments use three user-grouped validation folds within the run-0 development population, whereas each deep-learning cutoff model is trained once and evaluated on the canonical 15% user-grouped holdout, without the three-model ensemble. These diagnostic results assess relative degradation as recent information is withheld and are not directly comparable headline estimates. These choices were primarily due to computational and time constraints, as applying the full five-run repeated-holdout evaluation at every cutoff for every model would have required substantially more training time.

3.3.4. Temporal Stability Analysis

In addition to this we create a temporal split variable, `month_idx`, based on the expiry month. With this we can do a walk-forward validation. Thus models are trained on expiry months up to t, and then tested on $t + 1$. This provides a temporal stability check, and checks if models still perform on later months than they were trained on.

This walk-forward method is a temporal stability check, not an unseen-user evaluation. The tabular models use the development set of run-0, while the deep-learning model uses its canonical development split. A user with multiple events can therefore contribute an earlier event for training and later an event for testing. Only events happening before the test event will ever be used for training though, not the other way around. The purpose is therefore to assess temporal stability for future events, while the repeated user-grouped holdouts assess generalisation to unseen users.

3.3.5. Evaluation Metrics

The main evaluation that will be done is using Macro-F1. This is selected due to the class imbalance in the dataset and the equal importance of all classes in this case.

Accuracy alone is misleading due to a reasonable accuracy being achieved by predicting a majority `STABLE` class. Macro-F1 gives equal weight to each class, so performance on minority classes such as `CHURN` and `CONTRACTION` matters as much as performance on the majority class.

Macro-F1 is then calculated as the unweighted mean of the class-specific F1 scores:

$$\text{Macro-F1} = \frac{1}{K}\sum_{k=1}^{K} F1_k,$$

where $K = 4$ is the number of modellable classes.

We also report balanced accuracy, weighted-F1, and overall accuracy. Balanced accuracy is the average recall across classes:

$$\text{Balanced Accuracy} = \frac{1}{K}\sum_{k=1}^{K} \text{Recall}_k.$$

By doing this we measure whether or not the model identifies each class at a similar rate. The Weighted-F1 averages class specific F1 scores, and reflects the performance in context of the class distribution. The accuracy gives a valuable view of the overall share of correctly classified events, but is a complementary view only as it will be skewed by the class imbalance.

In addition to this we use confusion matrices for the selected model and key diagnostic analyses. Row-normalized matrices give a view of recall per class, and are helpful for interpreting which classes the models fail to classify.

The final model comparison is based primarily on holdout Macro-F1, with the other metrics used to interpret trade-offs between classes. Final results are reported as the mean and standard error across the five seeded user-grouped holdout runs described in Section 3.3.1.

To do the main model comparisons, the five user-grouped holdouts are treated as paired observations, as they use the same seed specific splits. Differences in run-level Macro-F1 are evaluated using two-sided paired t-tests across four pre-specified model contrasts, with Holm correction applied for multiple comparisons. With only five paired runs, inferential power and assumption checking are limited.

3.3.6. Computational Implementation

All experiments were implemented in Python and run locally on a Mac. Data preparation used `pandas`, `NumPy`, and `PyArrow` while the tabular models used `scikit-learn`, `LightGBM`, `XGBoost`, and `CatBoost`. The deep-learning models were implemented in

PyTorch. Intermediate datasets, tuning results, and trained models were cached where possible to avoid unnecessary recomputation.

The deep-learning experiments were the most computationally expensive part of the project. The main three-model ensemble required approximately 10.5 hours, while the larger robustness and time-before-renewal experiments required substantially longer. In total, the recorded deep-learning experiments accounted for approximately 105 hours of active computation, excluding failed runs and earlier exploratory versions.

The headline model comparison uses five repeated user-grouped holdouts to estimate variation across alternative user splits. Some secondary analyses use single-model fits or reduced evaluation procedures because repeating every diagnostic across five full runs would have substantially increased the computational cost. These analyses are therefore interpreted as model diagnostics rather than headline performance estimates.

4. Results

4.1. Dataset and Class Structure

The constructed expiry-event cohort contained 6,133,147 expiry events across 1,507,727 unique users. Of these, 61.7% passed our gate for sufficient pre-expiry data, leaving 3,784,500 events in the final modelling cohort.

For the final cohort, we see a strong class imbalance as expected, with 63.1% of events being STABLE, compared with 17.3% EXPANSION, 15.4% CONTRACTION, and 4.2% CHURN.

The full class distribution is shown in Supplementary Materials Section A.2.

This imbalance means that accuracy alone would be dominated by stable renewals, supporting the use of Macro-F1 as the main performance metric.

4.2. Predictive Performance

4.2.1. Baseline Performance

To gain perspective on the stronger predictive models we first ran some simple baselines. Firstly a majority-class classifier obtained a Macro-F1 of 0.1934, and a stratified random

prediction scored 0.2501. This allows us to conclude that all the substantive models learned predictive signals.

Next we did a model screening, testing a few different models. The bottom scorer was a simple Logistic Regression, scoring 0.4111, while LightGBM and XGBoost came in at 0.4386 and 0.4365 respectively.

Note that the model-screening runs were done without hyperparameter tuning and on a 200,000-row subsample. These results were purely to set a baseline and to test other models against the strongest candidates in LightGBM and XGBoost.

This performance is broadly consistent with previous churn-prediction work, as we also see gradient-boosted tree methods such as XGBoost and LightGBM among the strongest performances (Gregory, 2018; Imani et al., 2025). Previous work on the KKBox dataset specifically also showed strong performance from boosted-tree and ensemble approaches (Gregory, 2018; Wang et al., 2018). However, due to the original KKBox challenge and most reviewed literature being binary churn studies, the absolute scores here are not directly comparable.

4.2.2. Headline Repeated-Holdout Results

The results of the headline models that were trained and run five times on repeated 15% user holdouts are found in Table 3:

Table 3.: Holdout performance across the evaluated modelling approaches. Results are reported as mean ± standard error across five repeated user-grouped holdout runs.

Model	Macro-F1	Balanced Accuracy	Accuracy	Weighted-F1
Tuned DL ensemble	**0.5128 ± 0.0011**	0.5714 ± 0.0019	0.5940 ± 0.0020	**0.6077 ± 0.0013**
Direct XGBoost	0.4393 ± 0.0009	0.5709 ± 0.0003	0.4934 ± 0.0009	0.5205 ± 0.0008
Tuned two-stage classifier	0.4387 ± 0.0026	0.5720 ± 0.0004	0.4928 ± 0.0022	0.5201 ± 0.0019
Direct LightGBM	0.4379 ± 0.0008	**0.5728 ± 0.0005**	0.4913 ± 0.0008	0.5186 ± 0.0008
Two-stage arg-max	0.4150 ± 0.0006	0.5685 ± 0.0003	0.4717 ± 0.0006	0.5025 ± 0.0005
Two-stage ratio regression	0.3484 ± 0.0025	0.4742 ± 0.0006	**0.6124 ± 0.0028**	0.5350 ± 0.0016
Two-stage volume regression	0.3461 ± 0.0028	0.4709 ± 0.0004	0.6063 ± 0.0025	0.5292 ± 0.0018

The tuned deep-learning ensemble scored the highest Macro-F1, at 0.5128±0.0011, which was 0.0736 points over the strongest direct tabular model, XGBoost, which achieved

0.4393 ± 0.0009. Compared to XGBoost, this is an improvement of 16.7%. The ensemble's Macro-F1 ranged from 0.5103 to 0.5165 across the five holdout runs, showing consistency across repeated holdout sets.

The tuned deep-learning ensemble exceeded Direct XGBoost by a mean Macro-F1 difference of 0.0736, with a 95% paired confidence interval of $[0.0704, 0.0767]$. The ensemble outperformed XGBoost in all five runs, and the difference remained statistically significant after Holm correction ($t(4) = 64.36$, adjusted $p < 0.0001$).

The two strongest tabular models produced very similar results to each other. Direct XGBoost achieved a Macro-F1 of 0.4393, compared with 0.4379 for Direct LightGBM. XGBoost won in each of the five holdout runs, although the mean result was only marginally different.

The tuned two-stage classifier also performed similarly to the direct tabular models, achieving a Macro-F1 of 0.4387 ± 0.0026. This was 0.0006 below Direct XGBoost and 0.0008 above Direct LightGBM. We do see that the two-stage result had a larger standard error, and did not outperform direct classification across all five runs.

The mean Macro-F1 difference between the deep-learning ensemble and the two-stage classifier was 0.0742, with a 95% paired confidence interval of $[0.0658, 0.0825]$, and the paired comparison remained significant after Holm correction ($t(4) = 24.72$, adjusted $p < 0.0001$).

Both the regression-based methods performed substantially worse based on Macro-F1. The ratio-regression and volume-regression approaches achieved scores of 0.3484 and 0.3461, respectively. Despite this, they obtained the highest overall accuracies in Table 3, with the ratio-regression model reaching 0.6124. This highlights the class imbalance in the dataset, and reiterates the importance of Macro-F1 for our use case.

The tuned deep-learning ensemble did not outperform the tabular models on every reported metric. Its balanced accuracy of 0.5714 was similar to, but slightly below, the values obtained by LightGBM and the tuned two-stage classifier. At the same time, it produced substantially higher Macro-F1, accuracy and Weighted-F1. This shows that the final decision-weight tuning altered the balance between class precision and recall in favour of the primary Macro-F1 objective, rather than uniformly improving every metric.

4.3. Comparison of Modelling Methods

From the main headline results we see that the tuned deep-learning ensemble consistently outperforms tabular methods on Macro-F1. But individual modelling experiments also allow us to examine the results of splitting the problem into a two-stage, continuous regression, or a direct multiclass problem. This section compares the behaviour across the different approaches.

4.3.1. Direct Multiclass Classification

After initial screening we saw LightGBM and XGBoost as the strongest tabular models. By doing hyperparameter tuning within each repeated run, XGBoost achieved a mean Macro-F1 of 0.4393 ± 0.0009, while LightGBM scored 0.4379 ± 0.0008. Thus we see XGBoost being slightly higher, which was consistent across all 5 runs. XGBoost was therefore selected as the main direct model.

The difference between XGBoost and LightGBM was small, with the means only scoring 0.0013 apart. The paired comparison showed that this small difference was consistent across the five matched runs. XGBoost was higher in every run, with a mean paired difference of 0.0013 and a 95% confidence interval of $[0.0008, 0.0019]$ ($t(4) = 6.95$, Holm-adjusted $p = 0.0045$). It's worth noting that an improvement this small does not provide a true practical difference.

The class-level results for XGBoost display how certain classes are significantly harder to model. Mean F1 was 0.4166 for `CHURN`, 0.3194 for `CONTRACTION`, 0.6054 for `STABLE` and 0.4156 for `EXPANSION`. `CONTRACTION` was therefore the most difficult class for the direct model, while the majority `STABLE` class was predicted most effectively.

4.3.2. Two-Stage Classification

In the two-stage classifier we split the initial question of "Does the customer renew?" from the secondary "How do renewing customers change?". In the first renewal stage we score a strong performance. Using ROC-AUC and PR-AUC to look at the binary classifier results it scores ROC-AUC of 0.9566 and a PR-AUC of 0.9980. At a threshold of 0.5 the binary renew-versus-churn Macro-F1 was 0.6709. This very high PR-AUC should be interpreted in light of the strong imbalance though, with `RENEWED` being over

95% of the data in this case. It therefore does not imply equally strong identification of the minority `CHURN` class.

The movement stage is harder to model. When evaluated only among renewed users, the classifier achieved a Macro-F1 of 0.4691. The separate class F1 scores were 0.339 for `CONTRACTION`, 0.635 for `STABLE` and 0.433 for `EXPANSION`. This shows how the main challenge within the two-stage method (and probably the crux for the direct models) was separating between the post-renewal behaviours, rather than actually identifying churners.

Combining the two stages using the probability arg-max rule, the model scores a four-class Macro-F1 of 0.4150 ± 0.0006, which is lower than the direct classifiers. We then tuned the renewal decision threshold using the non-held-out training set, which increased the result to 0.4387 ± 0.0026, with a mean selected threshold of approximately 0.445.

Thus we see that threshold tuning recovered most of the lost performance. Even though this closed the gap, we do not see the two-stage method consistently outperform the direct classifier across the 5 runs, and it had a larger standard error. The paired analysis confirmed that there was no evidence of a systematic difference between the two approaches. Direct XGBoost exceeded the tuned two-stage classifier by only 0.0006 Macro-F1 on average, with a 95% confidence interval spanning zero, $[-0.0064, 0.0076]$ ($t(4) = 0.23$, Holm-adjusted $p = 0.8266$). XGBoost performed better in three runs and the two-stage model in two. Although we did not see a clear improvement in predictive performance, the two-stage method provides more interpretability in the results than the direct classifier.

4.3.3. Regression-Based Modelling

The two-stage regression model retrains the binary renewal classifier in stage 1, similar to the regular two-stage classifier, but for stage 2 replaces the classifier with a continuous regression of listening volume. Two separate approaches were made, one predicted post expiration listening volume, and then derived the ratio from this, and the other directly predicted the post-pre listening ratio. Then the predicted ratios were mapped to the 3 outcome classes using the same 0.50 and 1.50 thresholds used to construct the labels.

Neither regression model scored close to the stronger models according to Macro-F1. The ratio-regression model achieved 0.3484 ± 0.0025, while the volume-regression model

achieved 0.3461 ± 0.0028. These results were 0.09 points, or $\sim 20\%$ worse than the best direct classifiers.

For the ratio-regression model, the mean class F1 scores were 0.415 for `CHURN`, 0.038 for `CONTRACTION`, 0.762 for `STABLE` and 0.178 for `EXPANSION`. The volume-regression model achieved 0.418, 0.151, 0.759 and 0.056. This highlights how the continuous models were concentrated on the `STABLE` class and in general failed to predict the movement classes.

Direct classification, either in one or two stages, was therefore better suited for the four-class trajectory objective than predicting a continuous engagement quantity and applying fixed thresholds afterwards.

4.3.4. Deep-Learning Models

The deep-learning tests investigated whether raw temporal structure of listening behaviour contained predictive information beyond tabular summaries. The static-only neural network used the same features as the conventional tabular methods, while the hybrid model added a recurrent sequence branch over the 60 days of listening behaviour before expiry.

Table 4 is showing the deep-learning results on the first user-grouped holdout. These results are separate from the five-run headline evaluation and were produced on a single user-grouped holdout. They are used here to compare the different deep-learning configurations before the repeated evaluation.

Table 4.: Deep-learning performance on the single user-grouped holdout.

Model	**Macro-F1**	**Balanced Accuracy**	**Accuracy**	**Weighted-F1**
Static-only MLP	0.4293	0.5591	0.4962	0.5244
GRU and static features	0.4575	0.5906	0.5043	0.5295
Three-model ensemble, arg-max	0.4649	**0.5976**	0.5158	0.5412
Three-model ensemble, tuned	**0.5117**	0.5677	**0.6081**	**0.6163**

We see that the static-only network scored a Macro-F1 of 0.4293. After adding the raw 60-day listening sequence this increased to 0.4575, an absolute improvement of 0.0282, or approximately 6.6% relative to the static network. Thus we see that the temporal shape of pre-expiry listening activity has useful info that is not captured in the tabular features. As this comparison was done on a single user-grouped holdout, we see it as a diagnostic comparison and not a repeated estimate of improvement.

Just using arg-max to combine the ensemble achieves a Macro-F1 of 0.4649 on a single 15% holdout, outscoring the basic hybrid model.

We then use the development set to establish decision weights, maximising Macro-F1. Applying these weights to the single holdout run moved Macro-F1 from 0.4649 to 0.5117. At the same time, accuracy increased from 0.5158 to 0.6081, while balanced accuracy decreased from 0.5976 to 0.5677. Thus decision-weight tuning increased the Macro-F1 and combined class-level prediction, even though all metrics did not improve.

Finally, the full ensemble and decision-weighting run was repeated across the five repeated user-grouped holdouts. Doing this produced a mean Macro-F1 of 0.5128 ± 0.0011, closely matching the canonical holdout value of 0.5117.

4.4. Class-Level Performance of the Selected Model

When looking at the Macro-F1 of the highest performing model we are seeing how the results look across all four classes. To fully understand the model it is important to look at the class-level performances. We therefore use a row-normalized confusion matrix of the deep-learning ensemble. This matrix has independently calculated scores across 5 holdouts, which are then used to report the mean and standard error of each cell.

The analysis contains a total of 2.84M predictions, across the five runs. Figure 1 displays the mean recall confusion matrix for the final ensemble.

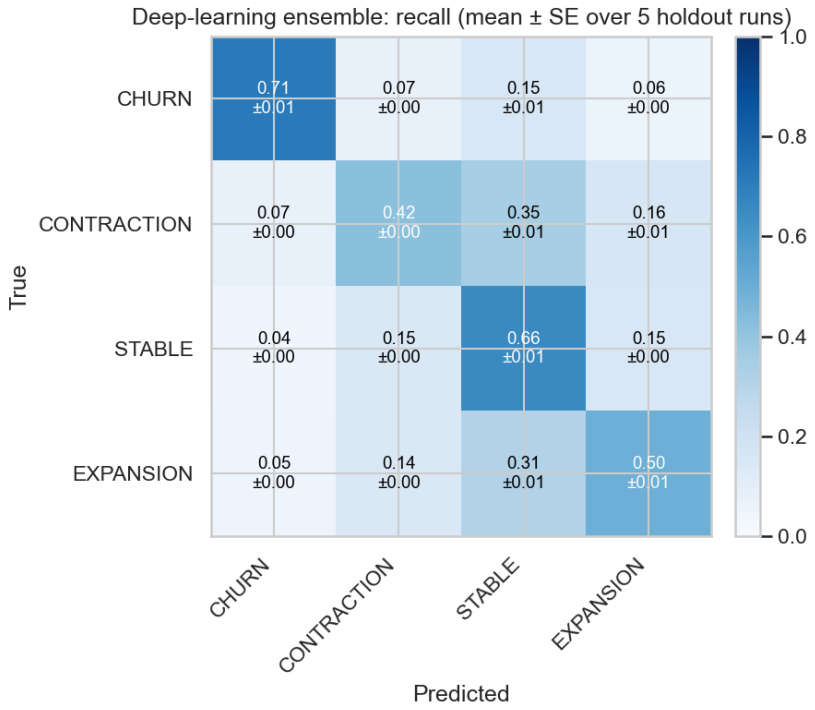

Figure 1.: Row-normalised confusion matrix for the tuned deep-learning ensemble. Each cell reports mean recall ± standard error across five repeated user-grouped holdout runs.

The model performs best when predicting `CHURN`, correctly identifying $71.4\% \pm 0.8\%$ of churn events. For `STABLE` and `EXPANSION` it achieves a recall of $65.5\% \pm 0.6\%$ and $49.6\% \pm 0.8\%$ respectively. As before, `CONTRACTION` is the most difficult class to predict, and has a recall of $42.1\% \pm 0.5\%$.

Looking at the errors we see the strongest pattern being movement towards `STABLE`. Of the true `CONTRACTION` events, $34.7\% \pm 0.7\%$ were predicted as `STABLE`. Similarly, $31.2\% \pm 0.8\%$ of true `EXPANSION` events were classified as `STABLE`. As we see from the two-stage model, this one also is often successful at identifying that a user would renew, but struggles to determine what type of renewal.

For `CHURN` we see fewer errors overall, approximately $15.2\% \pm 0.5\%$ of churn events were classified as `STABLE`, while $7.2\% \pm 0.4\%$ were classified as `CONTRACTION` and $6.2\% \pm 0.3\%$ as `EXPANSION`. Interestingly, we see a similar error towards `CONTRACTION` and `EXPANSION` here, indicating that when the model is wrong about `CHURN` it can be very wrong. The

fairly high CHURN recall is consistent with the two-stage model, where we also saw stronger results for the CHURN prediction.

The confusion matrix shows that although Macro-F1 improved, the four classes did not all meaningfully improve by the same magnitude. CHURN and STABLE are predicted well, while CONTRACTION remains a difficult problem to track. The main remaining classification challenge was therefore distinguishing moderate negative movement from a stable renewal.

4.5. Time-before-renewal Analysis

The main models, and all results in Section 4.2 use all information available to them to maximise prediction results. In a commercial setting, knowing ahead of time is what really matters, as this gives time to try and counteract negative outcomes. For a consumer-based product like KKBox, a few days or a week ahead of time might be enough, while for a huge corporate contract months ahead might be required to alter the outcome.

To investigate how the different results handle this we rerun the models with prediction cutoffs at 7, 14, 30, and 60 days ahead of the expiration, and don't allow the model to train on this data to simulate predicting renewal outcomes ahead of the renewal. Outcome labels remain fixed, while data available before the expiration shifts. For each cutoff all time-varying features are rebuilt using only the information available by $E - c$.

4.5.1. Performance of the Selected Model

As the deep-learning model scored the highest for the regular runs we focus on that as the selected model here. In the regular runs we used a constructed 60-day listening sequence for the deep-learning models. This 60-day approach is still used, but now slides. For example, the last cutoff has a panel of $E - 120$ to $E - 61$.

The model was retrained from scratch at each cutoff using the same information restriction during training and evaluation. Static behavioural features, transaction-history, and subscription-history variables were rebuilt at each cutoff using only information available by $E - c$. Demographic variables which are not time-varying remained available.

Table 5.: Deep-learning performance under fixed-length sliding sequence cutoffs using one retrained model per cutoff.

Decision point	Sequence window	Coverage	Macro-F1	Retention
At expiry	$E-60$ to $E-1$	100.0%	**0.5092**	100.0%
7 days before	$E-67$ to $E-8$	99.7%	0.4834	94.9%
14 days before	$E-74$ to $E-15$	99.6%	0.4673	91.8%
30 days before	$E-90$ to $E-31$	96.2%	0.4018	78.9%
60 days before	$E-120$ to $E-61$	93.2%	0.3793	74.5%

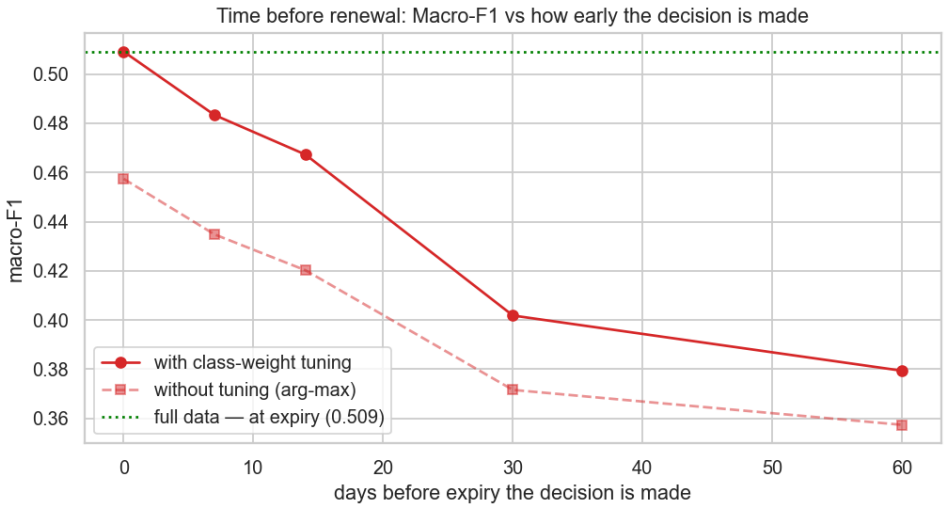

Figure 2.: Macro-F1 of the single retrained deep-learning model at each prediction cutoff. The sequence remains fixed at 60 days, while the time-varying static features are rebuilt at the relevant prediction point. The solid line uses the tuned decision weights and the dashed line uses unadjusted arg-max.

In Table 5 we see the results based on cutoff. We also display the Coverage, which is the share of events with at least one non-zero listening observation in the relevant 60-day sequence, and Retention, which is how much of the peak Macro-F1 is still scored at cutoff.

For the first 7-day cutoff, we see a reduced Macro-F1 from 0.5092 to 0.4834, an absolute reduction of 0.0258. Predicting 14 days before expiry produced a score of 0.4673, 0.0161

below the seven-day result. The model therefore retained approximately 91.8% of its full-information performance when predicting two weeks before expiry.

Between 14 and 30 days we see a steeper drop-off in results. At the 30-day cutoff, Macro-F1 decreased to 0.4018, an absolute reduction of 0.1074 from the at-expiry result. At 60 days before expiry, performance decreased further to 0.3793. This was an absolute reduction of 0.1299 from the at-expiry result, meaning the model retained 74.5% of its full-information performance.

Thus we see that the decline is not linear. During the first two weeks the model retains most of its quality. Then we see a steep drop-off between 14 and 30, while between 30 and 60 the drop flattens again. This could indicate that for the KKBox product, the last month of listening data is the most valuable to predict renewal trajectory. It is also worth noting that at the 30-day cutoff the model does not see any data that is used in the pre-expiry part of defining the outcome classification. This is a product of the dataset design, and could impact the models in ways a real-world scenario wouldn't.

It is also worth noting that the sequence coverage dropped at that gap. Coverage remained above 99% at seven and 14 days, but fell to 96.2% at 30 days and 93.2% at 60 days. Events without observed listening in the relevant sequence were still retained and represented using zero-filled sequence inputs.

Figure 2 also presents the result obtained using the unadjusted arg-max prediction rule. The tuned decision weights produced higher Macro-F1 at every cutoff, but both prediction rules followed the same general decay pattern.

We note that the at-expiry cutoff presented here is slightly lower than the main performance model presented earlier in the paper, at 0.5092 compared to 0.5128. The time-before-renewal experiments were run as model-specific diagnostics rather than using the full five-run repeated-holdout protocol. The tabular cutoff models were evaluated using user-grouped out-of-fold predictions on the development population, while the deep-learning cutoff used a single retrained model at each time point rather than the full three-model ensemble. Thus these results are not entirely comparable with the main headline results, but are more intended as relative results across cutoffs within each modelling approach.

4.5.2. Comparison Across Model Formulations

We also did the time-before-renewal analysis for both XGBoost as the direct classifier and the LightGBM-based two-stage classifier. In Table 6 we compare the decay results.

Table 6.: Macro-F1 at different prediction cutoffs across the main modelling formulations. Tabular results use three-fold user-grouped cross-validation, while DL results use one retrained model per cutoff; cross-model comparisons are therefore descriptive.

Decision point	Direct XGBoost	Two-stage LightGBM	DL hybrid
At expiry	0.4380	0.4375	**0.5092**
7 days before	0.4221	0.4178	**0.4834**
14 days before	0.4066	0.4025	**0.4673**
30 days before	0.3769	0.3728	**0.4018**
60 days before	0.3593	0.3566	**0.3793**

The Direct XGBoost and two-stage LightGBM classifiers followed almost identical decay curves. At the 14-day cutoff, XGBoost retained 92.8% of its at-expiry Macro-F1, while the two-stage model retained 92.0%. At 60 days, they retained 82.0% and 81.5%, respectively. So using the two-stage formulation did not have a meaningful impact on the relative decline in performance ahead of time.

The deep-learning model remained numerically strongest at every cutoff, although the DL and tabular results use different diagnostic evaluation protocols and the absolute differences should therefore be interpreted descriptively. The performance gap became substantially smaller at the 30- and 60-day cutoffs. Although we see that the 30- and 60-day cutoff DL hybrid models still outperform the Direct XGBoost and two-stage LightGBM models at the same cutoffs, they no longer outperform the at-expiry XGBoost and two-stage results.

4.5.3. Renewal and Movement Stage Sensitivity

By running the two-stage model across different time steps we gain the advantage of specifying where the model starts to fall apart. In Table 7 we show the Macro-F1 of the different stages across the prediction cutoffs.

Table 7.: Time-before-renewal performance for the renewal and movement components of the two-stage classifier.

Decision point	Renewal-stage F1	Movement-stage F1	Derived four-class F1
At expiry	0.6727	0.4695	0.4375
7 days before	0.6488	0.4593	0.4178
14 days before	0.6384	0.4455	0.4025
30 days before	0.6217	0.4156	0.3728
60 days before	0.6094	0.3995	0.3566

We see that the renewal/churn stage results decrease across the entire test. Renewal-stage Macro-F1 decreased from 0.6727 at expiry to 0.6094 at 60 days before expiry, a reduction of 0.0633. This indicates that having the most recent data available was important for the renewal stage as well, although it reduced at a lower rate than the overall model. Thus things like transaction history, subscription state, auto-renew toggle, and user metadata can be enough to model CHURN from renewals, although not at as high an accuracy.

On the other hand the movement stage was also sensitive to days before renewal cutoffs, with Macro-F1 decreasing from 0.4695 at expiry to 0.4455 at 14 days, 0.4156 at 30 days, and 0.3995 at 60 days. The total reduction between expiry and the 60-day cutoff was 0.0700.

The primary driver for the worse results at the longer cutoffs was the reduced ability to distinguish between `CONTRACTION`, `STABLE`, and `EXPANSION`. At the 60-day cutoff, renewal-stage Macro-F1 had declined by 9.4% from its at-expiry result, while movement-stage Macro-F1 had declined by 14.9%. Thus both stages were affected by the earlier cutoff, but the movement stage deteriorated substantially more.

Overall the time-before-renewal analysis shows that renewal trajectory can be predicted reasonably accurately 1-2 weeks before expiry. A 14-day prediction retained approximately 91.8% of the deep-learning cutoff model's full-information Macro-F1, while providing substantially more lead time than a prediction made immediately before expiry. For a consumer-based product like KKBox this might be plenty of time to counteract, as often subscription decisions are made month to month on these types of products. Predictions 30-60 days ahead of time were informative, but had a much larger drop-off

in performance, with the deep-learning model retaining 74.5% of its at-expiry performance at the 60-day cutoff. From a two-stage perspective we did see that the drop-off is the most significant at the second stage, while a binary churn classifier still performs decently at a longer cutoff.

4.6. Temporal Stability

4.6.1. Walk-Forward Validation

We do the five repeated holdout tests to analyse performance across different groups of users, but it does not test whether the results are consistent across each month. A model could therefore use early months to generalise towards the later months. We therefore do a walk-forward validation to test temporal stability, where the models were trained on all expiry months up to month t, and evaluated on the following month.

Table 8.: Walk-forward Macro-F1 across later expiry months. Tabular models use the run-0 development set and DL uses its canonical development split and a single hybrid model.

Test month	Direct XGBoost	Two-stage	DL hybrid
September 2016	0.4422	0.4526	0.4448
October 2016	0.4419	0.4488	0.4502
November 2016	0.4528	0.4571	0.4695
December 2016	0.4309	0.4321	0.4440
January 2017	0.4454	0.4456	0.4524

We see that the performance stays relatively stable across all three approaches. Direct XGBoost ranged from 0.4309 to 0.4528, the two-stage classifier ranged from 0.4321 to 0.4571, and the deep-learning model ranged from 0.4440 to 0.4695. The deep-learning model achieved the strongest result in four of the five test months.

All models produced their lowest scores in December 2016, before improving again. There was no real decline as the test period moved. Thus the predictive relationships were reasonably stable across all five test months.

The deep-learning experiment here was done on the basic single hybrid GRU, and not the enhanced three-model ensemble. These results should therefore not be compared to the top model results, but are to check stability for the model itself.

4.7. Model Interpretation

The main deep-learning ensemble does not provide a simple feature-importance ranking. We therefore use the XGBoost direct model to look at which tabular features were the strongest. Gain-based feature importance was calculated for the five holdouts, normalised within each run, and then averaged.

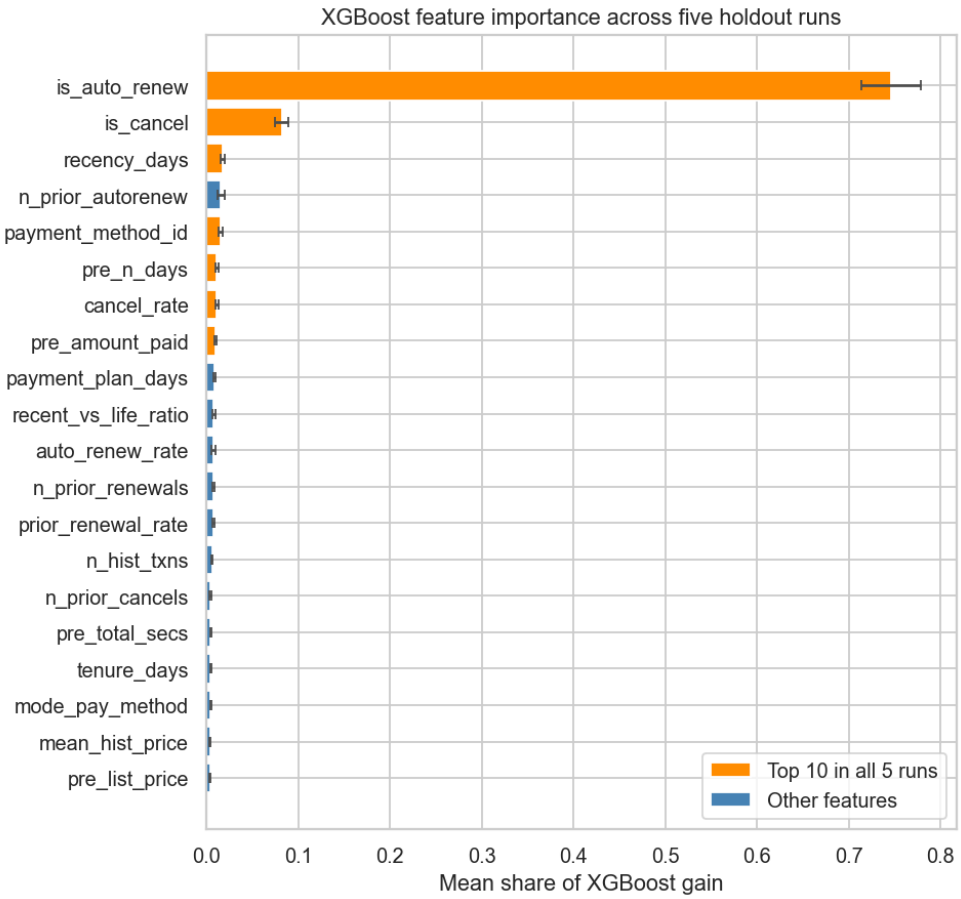

Figure 3.: Gain-based XGBoost feature importance averaged across the five repeated user-grouped holdout runs. Error bars show ± one standard error across runs. Highlighted features appeared within the top 10 in all five runs.

The auto-renew indicator was by far the strongest XGBoost feature, with a mean gain

share of 0.746, and ranked first in all five holdout runs. The cancellation indicator was second, with a mean gain share of 0.082, and was also among the five highest-ranked features in every run. After this, transaction recency, payment method, prior auto-renewal history, pre-expiry active listening days, cancellation history, and amount paid were among the strongest features. Seven features appeared within the top 10 in all five runs. Thus the broad feature-importance ranking was stable across the holdouts.

We also looked at how the two dominant binary features related to the response classes. In events where auto-renew was disabled, we saw that 17.8% were CHURN, compared with 1.3% where auto-renew was enabled. Cancellation status showed a similar relationship, when the cancellation indicator was active, 16.2% of events were CHURN and 33.1% were CONTRACTION, compared with 4.0% and 15.0%, respectively, when it was inactive. These are entirely descriptive relationships, and no causal study was done.

Despite its dominant gain importance, auto-renew status alone was not sufficient for the four-class prediction task. A model trained only on `is_auto_renew` achieved a Macro-F1 of 0.1478 ± 0.0002 across the five holdout runs, compared with 0.4393 ± 0.0009 for the full XGBoost model. Thus, although auto-renew was a dominant feature, it does not explain the full predictive performance of the model alone, recovering only 33.7% of the full-model Macro-F1 when used as the only feature.

The importance ranking indicates that subscription state and historical transaction behaviour were central to the tabular model. Recent listening behaviour also contributed through features such as pre-expiry active listening days and the ratio between recent and longer-term listening intensity. Separately, the deep-learning experiments showed that preserving the raw daily listening sequence improved performance over the static-only neural network.

These importance values are specific to XGBoost, do not show the direction of an effect, and should not be interpreted causally. They also do not directly explain the predictions of the final deep-learning ensemble.

5. Discussion

In this dissertation we looked into whether pre-expiration behaviour, transaction history, and other customer metadata could predict four post-expiry renewal trajectories: `CHURN`, `CONTRACTION`, `STABLE`, and `EXPANSION`. The results show that the out-

comes have predictable structure beyond traditional binary churn/renewal outcomes. The tuned deep-learning ensemble achieved the strongest performance, with a Macro-F1 of 0.5128 ± 0.0011, while the time-before-renewal analysis showed that approximately 92% of at-expiry performance was retained when predictions were made 14 days before expiry.

The four-class formulation provides a more detailed account of customer renewal than the binary churn framing commonly used in previous work (Altinisik and Yilmaz, 2019; de Lima Lemos et al., 2022; Momin et al., 2020). A renewed customer who uses less, or in a different setting maybe buys a lower subscription, should not be treated the same as an upgrade, which is what a binary churn model does. The current results indicate that these richer states are predictable, although distinguishing between closely related post-renewal behaviours is more difficult than separating churn from renewal.

From the deep-learning results we saw that the temporal structure of listening behaviour contains information not fully captured by aggregated tabular summaries. Direct XGBoost and the tuned two-stage classifier achieved almost identical results, and the paired analysis found no evidence of a systematic difference between them. This means that separating renewal from post-renewal movement did not provide a clear predictive advantage. From the two-stage model we saw that predicting the renewal was easier than predicting what class of renewal. The regression approaches performed considerably worse because their predictions concentrated around the central `STABLE` region. Although this produced high overall accuracy, it resulted in weak identification of `CONTRACTION` and `EXPANSION`. Within the direct XGBoost model, transaction-state information was particularly important, auto-renew status ranked as the strongest feature in all five repeated fits, although a model using this feature alone performed substantially worse than the complete feature set.

From the class-level analysis we further saw that performance was uneven across the four outcomes. `CHURN` and `STABLE` had the highest recall, while `CONTRACTION` remained the most difficult class. Both contractions and expansions often ended up as stable. Part of this could be a true difficulty in the prediction. There is also an element that this may partly reflect the construction of the labels. The movement classes are produced by thresholding an underlying continuous listening ratio at 0.50 and 1.50, meaning that events close to either boundary may have similar behavioural profiles despite receiving different labels. As this is a consumer-level product there are also likely usage noise and seasonality in play. A user with an upcoming exam may listen extensively over a week,

and then not at all the month after, regardless of actual sentiment around the product or any true intentional shift in behaviour.

The time-before-renewal analysis provides us with some of the strongest real world applications of the project. We saw that performance only moderately declined at 7- and 14-day cutoffs before the expiration, and then fell sharply to the 30-day cutoff, before a smaller further decline at 60 days. The two-stage decomposition showed that this decline was caused by reduced performance in both stages, although the movement-stage performance fell more at the longer cutoffs. From a commercial perspective this is an important differentiation. Often task one is customer retention, which we see we can predict early with less of an accuracy decline, and then a follow up behavioural- change once the model gets more accurate closer to expiration. For a consumer subscription service like KKBox it is not shocking that signals older than a couple of weeks get worse, as decisions at consumer level are inherently done quicker. If these methods were applied to enterprise-level SaaS contracts one may need to predict 3-6 months ahead of time, but could also reasonably suspect more stability in the patterns well ahead of time, as often enterprise contracts may span 1, 3, or even 5 years. Thus the actual horizon cutoff vs accuracy trade-off would vary on a product-by-product basis.

There are several limitations to this dissertation. Firstly, the outcome classes are self-constructed, and not true commercial labels. The 0.50 and 1.50 thresholds are interpretable but are subjective modelling choices, driven by domain knowledge and knowingly creating a class imbalance, as this often is the commercial setting. Alternative choices would impact results and prediction difficulty. The seven-day pre-expiry activity gate also impacts the models, as removing customers with sparse activity means the final model does not contain every single user, but the trade-off was made to not have too many users that were "noise" in the data. It's also important to note that the results are based on one music-streaming service and a six-month expiry cohort, meaning we do not normalise against seasonality, nor validate across multiple datasets. The final ensemble scores the highest, but is also less interpretable than the tabular models, and XGBoost feature importance cannot directly explain the sequence representations learnt by the neural network. Feature engineering was also largely domain-driven. We looked at correlations, class-conditional relationships, and the stability of XGBoost feature importance, but we did not do a systematic analysis of interactions or conditional relationships across the full feature set. The paired statistical comparisons were based on only five repeated holdout runs, which limits inferential power and the ability to assess distributional assumptions. Due to computational and time constraints, some secondary analyses used reduced evaluation procedures, including 200,000-event devel-

opment subsamples, single model runs, and a single hybrid deep-learning model rather than the full ensemble, instead of repeating the complete five-holdout evaluation for every experiment. Thus these secondary results are not intended for headline comparison, but as diagnostic analyses of model behaviour.

Future work on this specific dataset could test whether the same conclusions are consistent under alternative movement thresholds or activity gating. Although more interesting future work would be applying these same methods on different subscription datasets. Especially larger, slower-changing, commercial datasets would provide strong evidence of generalisability should the results remain the same. Further modelling could also focus specifically on separating `CONTRACTION` from `STABLE`, potentially using longer behavioural histories, trend-specific features, or alternative sequence architectures. Then, once these models are built actual intervention testing or A/B testing while using the outcome classifications would indicate if there is commercial gain in trying to predict and counteract renewal outcomes using methods like these.

Overall, the findings show that subscription expirations should not be treated only as a binary question of churn, and show that it is possible to predict with meaningful accuracy more advanced outcomes. Pre-expiry behaviour can also provide information about how customer engagement will change after renewal. Sequence-based modelling produced the strongest predictions, and many of the signals used are available well ahead of time of the expiration, even for a consumer-based dataset. The main unresolved challenge is not identifying renewal itself, but predicting the direction of post-renewal behaviour, particularly whether a renewed customer will contract or remain stable.

6. Endmatter

Data Availability

The data are publicly available through the WSDM–KKBox Churn Prediction Challenge on Kaggle (2017).

Code Availability and Reproducibility

Code and instructions for reproducing the reported analyses are available in the project GitHub repository, which contains the data-processing, EDA, modelling and statistical-analysis notebooks and shared utilities.

References

Fehim Altinisik and Hasan Huseyin Yilmaz. Predicting customers intending to cancel credit card subscriptions using machine learning algorithms: A case study. In *2019 11th International Conference on Electrical and Electronics Engineering (ELECO)*. IEEE, 2019. doi: 10.23919/ELECO47770.2019.8990563.

Eva Ascarza and Bruce G. S. Hardie. A joint model of usage and churn in contractual settings. *Marketing Science*, 32(4):570–590, 2013. doi: 10.1287/mksc.2013.0786.

Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001. doi: 10.1023/A:1010933404324.

Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002. doi: 10.1613/jair.953.

Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.

Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder–decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.

Renato Alexandre de Lima Lemos, Thiago Christiano Silva, and Benjamin Miranda Tabak. Propension to customer churn in a financial institution: a machine learning approach. *Neural Computing and Applications*, 34:11751–11768, 2022. doi: 10.1007/s00521-022-07067-x.

Yumo Dong, Edward W. Frees, Fei Huang, and Francis K. C. Hui. Multi-state modelling of customer churn. *ASTIN Bulletin*, 52(3):735–764, 2022. doi: 10.1017/asb.2022.18.

Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.

Bryan Gregory. Predicting customer churn: Extreme gradient boosting with temporal data. *arXiv preprint arXiv:1802.03396*, 2018. URL https://arxiv.org/abs/1802.03396.

Mehdi Imani, Majid Joudaki, Ali Beikmohammadi, and Hamid Reza Arabnia. Customer churn prediction: A systematic review of recent advances, trends, and challenges in machine learning and deep learning. *Machine Learning and Knowledge Extraction*, 7(3):105, 2025. doi: 10.3390/make7030105.

Shylu John, Bhavin Shah, Varun Dixit, and Amol Wani. An integrated approach to

renew software contract using machine learning. *Journal of Business Analytics*, 4(1): 14–25, 2021. doi: 10.1080/2573234X.2020.1863749.

Kaggle. WSDM - KKBox's Churn Prediction Challenge. https://www.kaggle.com/competitions/kkbox-churn-prediction-challenge, 2017. Accessed for dissertation data source.

Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, pages 3146–3154, 2017.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

Saifil Momin, Tanuj Bohra, and Purva Raut. Prediction of customer churn using machine learning. In *EAI International Conference on Big Data Innovation for Sustainable Cognitive Computing*, pages 203–212. Springer, 2020. doi: 10.1007/978-3-030-19562-5_20.

Liudmila Prokhorenkova, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. Catboost: Unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

Van-Hieu Vu. Predict customer churn using combination deep learning networks model. *Neural Computing and Applications*, 36:4867–4883, 2024. doi: 10.1007/s00521-023-09327-w.

Zhinan Wang, Wenming Xiao, and Jun Wang. A practical pipeline with stacking models for kkbox's churn prediction challenge. In *Proceedings of the WSDM Cup 2018*, 2018. URL https://wsdm-cup-2018.kkbox.events/pdf/7_A_Practical_Pipeline_with_Stacking_Models_for_KKBOXs_Churn_Prediction_Challenge.pdf.

A. Supplementary Materials

A.1. Class Assignment Flowchart

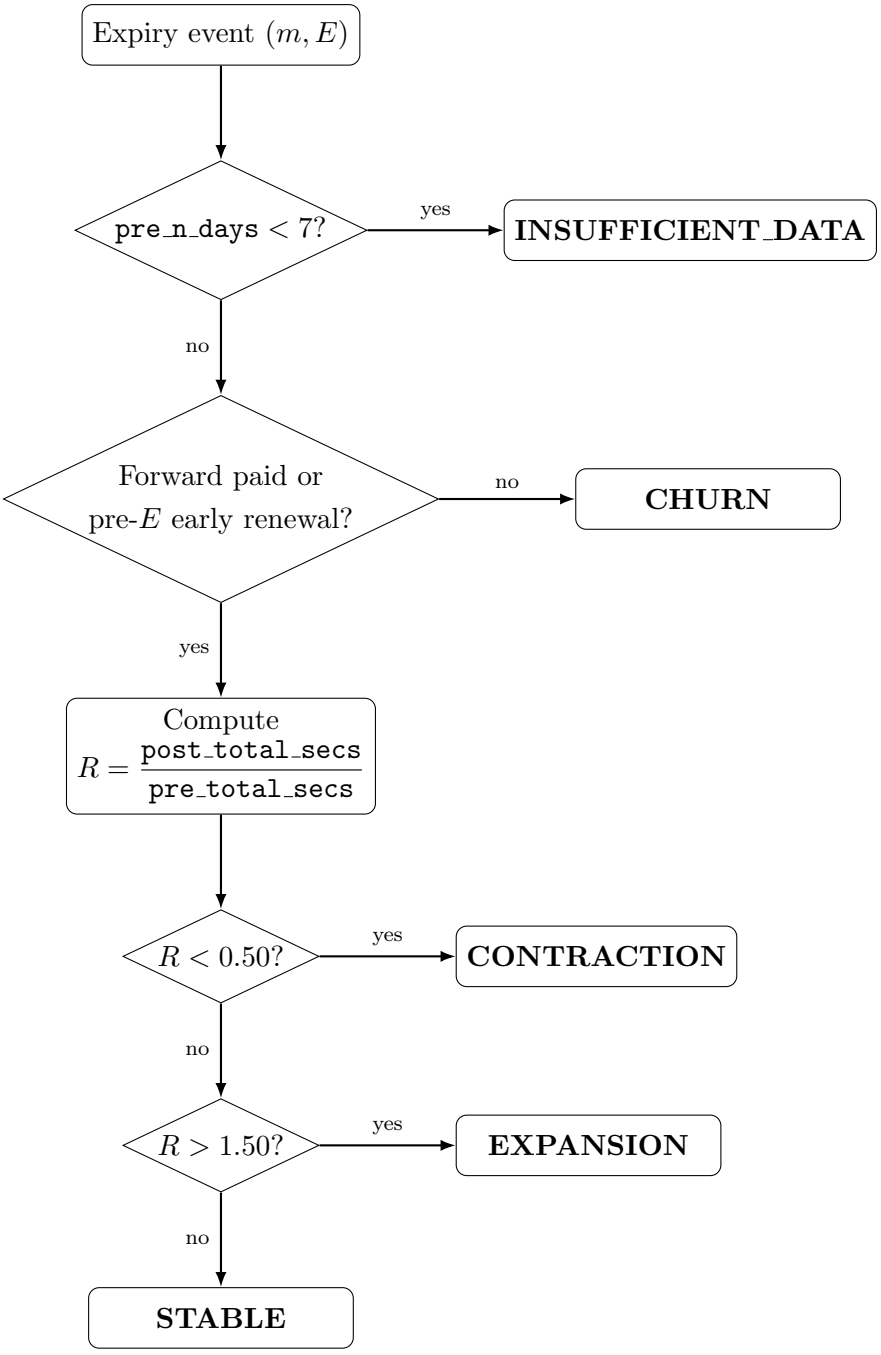

Figure 4.: Flowchart for assigning renewal-trajectory labels to expiry events.

A.2. Class Distribution

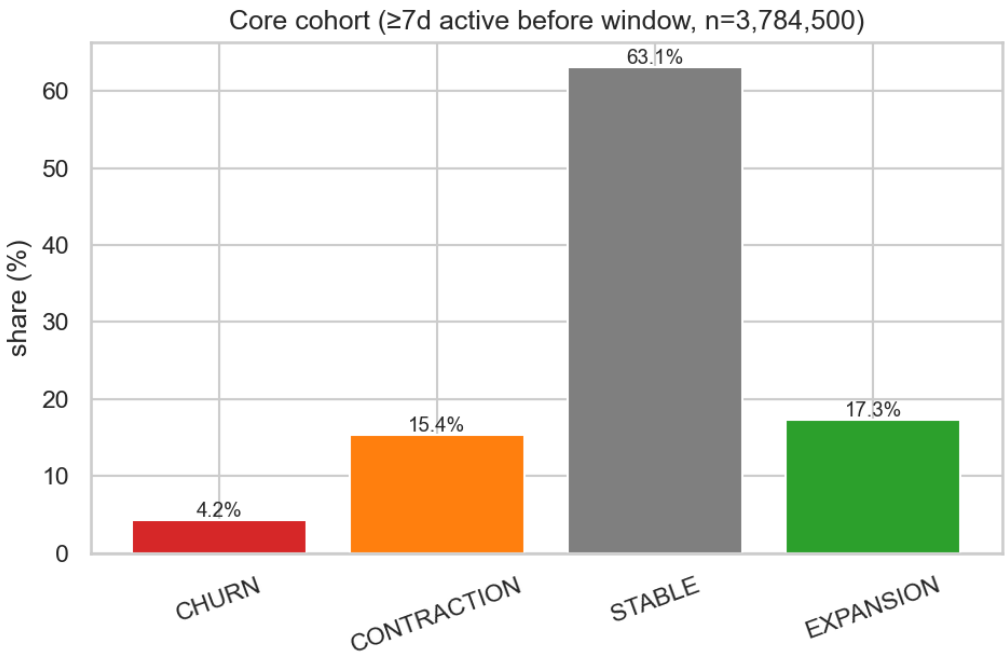

Figure 5.: Distribution of outcome classes in the final modellable cohort.

A.3. Hyperparameter Search Spaces

The main hyperparameter search spaces and tuning procedure are summarised below.

Table 9.: Hyperparameter search space for LightGBM.

Hyperparameter	**Candidate values**
`n_estimators`	300, 500, 800
`learning_rate`	0.03, 0.05, 0.10
`num_leaves`	31, 63, 127, 255
`min_child_samples`	20, 50, 100, 200
`colsample_bytree`	0.70, 0.85, 1.00
`subsample`	0.70, 0.85, 1.00
`reg_lambda`	0.0, 1.0, 5.0

Table 10.: Hyperparameter search space for XGBoost.

Hyperparameter	Candidate values
n_estimators	300, 500, 800
learning_rate	0.03, 0.05, 0.10
max_depth	4, 6, 8, 10
min_child_weight	1, 5, 20
subsample	0.70, 0.85, 1.00
colsample_bytree	0.70, 0.85, 1.00
reg_lambda	0.0, 1.0, 5.0
gamma	0.0, 0.5, 2.0

A.4. Model Selection and Tuning Procedure

Table 11.: Summary of the hyperparameter-selection procedure.

Component	Procedure
Development subsample	Approximately 200,000 events drawn from the development users for each repeated-holdout run.
Cross-validation	Three-fold user-grouped cross-validation.
LightGBM search	16 randomly sampled configurations for the direct and renewal models, and 24 for the movement and regression models, using the search space in Table 9.
XGBoost search	16 randomly sampled configurations from the search space in Table 10.
Selection criterion	Mean validation Macro-F1 across the three folds.
Repeated holdouts	Five user-grouped holdout runs using seeds 42–46; hyperparameters were selected independently within each run.
Final fitting	The selected configuration was refitted on all development users and evaluated once on that run's untouched holdout.

Table 12.: Models included in the initial tabular screening stage.

Model family	Role
Logistic Regression	Linear baseline
Random Forest	Bagged tree ensemble
HistGradientBoosting	Histogram-based gradient boosting
CatBoost	Gradient-boosted trees with categorical-feature handling
LightGBM	Gradient-boosted tree candidate for further tuning
XGBoost	Gradient-boosted tree candidate for further tuning

A.5. AI Usage Statement

All work in this dissertation is my own, but generative AI tooling has been used to assist me:

- **ChatGPT** was used for search, brainstorming, technical discussion, and debugging/reviewing parts of the code and modelling logic. The project itself came from my own day-to-day experience working in a commercial Data Science setting.
- **ChatGPT** was also used for proofreading, LaTeX formatting of tables, TikZ, and general language edits. The writing and arguments are my own, with AI used as a proofreading and formatting tool.
- **VS Code GitHub Copilot** was used as a pair-programming tool for code completion, debugging, refactoring, cleanup, and final repository preparation. The modelling approach and core implementation were developed by me, with AI tools used to assist and improve the code.

The research problem itself relates directly to my day-to-day work experience as a commercial Data Scientist, and the research questions, methods, modelling decisions, interpretation of results, and conclusions were developed independently by me. I reviewed and remain responsible for all submitted code and written content.